

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>

#define MAX_ARGS 64
#define BUF_SIZE 512

// 1. PATH RESOLVER: Searches system PATH directories for executable binaries
char* resolve_command_path(const char *cmd) {
    // If command already contains a path (e.g. ./program or /bin/ls)
    if (strchr(cmd, '/')) {
        if (access(cmd, X_OK) == 0) {
            return strdup(cmd);
        }
        return NULL;
    }

    // Retrieve PATH environment variable
    char *path_env = getenv("PATH");
    if (!path_env) return NULL;

    // Duplicate PATH string to avoid modifying original environment pointer
    char *path_copy = strdup(path_env);
    char *dir = strtok(path_copy, ":");
    char full_path[BUF_SIZE];

    while (dir != NULL) {
        snprintf(full_path, sizeof(full_path), "%s/%s", dir, cmd);

        // Verify file existence and execution permissions (X_OK)
        if (access(full_path, X_OK) == 0) {
            free(path_copy);
            return strdup(full_path); // Executable found
        }
        dir = strtok(NULL, ":");
    }
}
```

```

        dir = strtok(NULL, ":");
    }

    free(path_copy);
    return NULL; // Command missing or not executable
}

// 2. PROCESS SYNCHRONIZATION: Launches child process and monitors with waitpid()
void execute_and_monitor(char *args[]) {
    if (args[0] == NULL) return;

    // Locate executable via PATH variable resolution
    char *resolved_path = resolve_command_path(args[0]);

    if (!resolved_path) {
        printf("Error: Command '%s' not found or permission denied.\n", args[0]);
        return;
    }

    printf("[PATH Resolver] Resolved '%s' -> '%s'\n", args[0], resolved_path);

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        free(resolved_path);
        return;
    } else if (pid == 0) {
        // Child Process: Execute resolved path
        execv(resolved_path, args);
        perror("Execv failed");
        exit(EXIT_FAILURE);
    } else {
        // Parent Process: Monitor Child using waitpid()
        int status;
        printf("[Parent] Monitoring Child Process PID: %d\n", pid);
    }
}

```

```

        if (waitpid(pid, &status, 0) == -1) {
            perror("waitpid failed");
        } else {
            // Inspect Child Exit Status & Signals
            if (WIFEXITED(status)) {
                printf("[Parent] Child PID %d exited normally with code: %d\n", pid, WEXITSTATUS(status));
            } else if (WIFSIGNALED(status)) {
                printf("[Parent] Child PID %d killed by signal: %d\n", pid, WTERMSIG(status));
            }
        }
    }

    free(resolved_path);
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nPATH Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        // Remove newline
        input_buf[strcspn(input_buf, "\n")] = 0;

        if (strcmp(input_buf, "exit") == 0) break;

        // Simple space-delimited argument parser
        char *args[MAX_ARGS];
        int arg_count = 0;
        char *token = strtok(input_buf, " ");

        while (token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " ");
        }
        args[arg_count] = NULL;

```

```

        if (arg_count > 0) {
            execute_and_monitor(args);
        }
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano path_waitpid.c
abhis@MIKHAIL-37:~$ gcc -g -Wall path_waitpid.c -o path_waitpid
abhis@MIKHAIL-37:~$ ./path_waitpid
```

```
PATH Shell > /bin/date
[PATH Resolver] Resolved '/bin/date' -> '/bin/date'
[Parent] Monitoring Child Process PID: 1067
Tue Sep 29 09:00:10 UTC 2026
[Parent] Child PID 1067 exited normally with code: 0
```

```
PATH Shell > nonexistentcommand123
Error: Command 'nonexistentcommand123' not found or permission denied.
```

```
PATH Shell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./path_waitpid
==1079== Memcheck, a memory error detector
==1079== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1079== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1079== Command: ./path_waitpid
==1079==
```

```
PATH Shell > exit
==1079==
==1079== HEAP SUMMARY:
==1079==    in use at exit: 0 bytes in 0 blocks
==1079==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1079==
==1079== All heap blocks were freed -- no leaks are possible
==1079==
==1079== For lists of detected and suppressed errors, rerun with: -s
==1079== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```